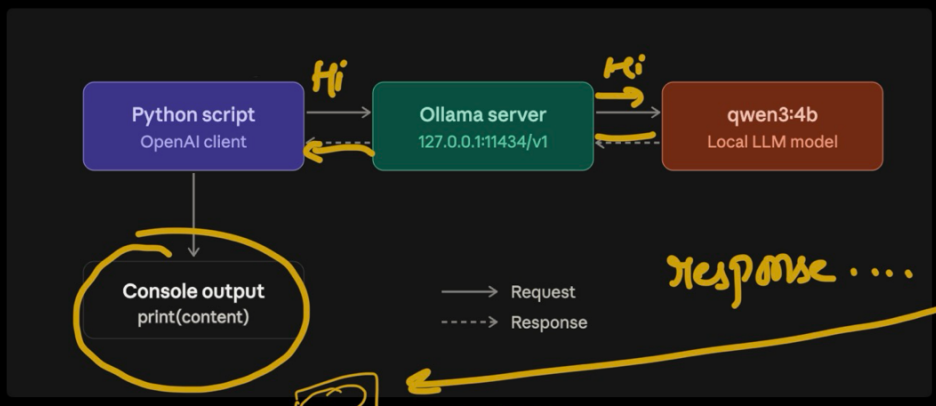
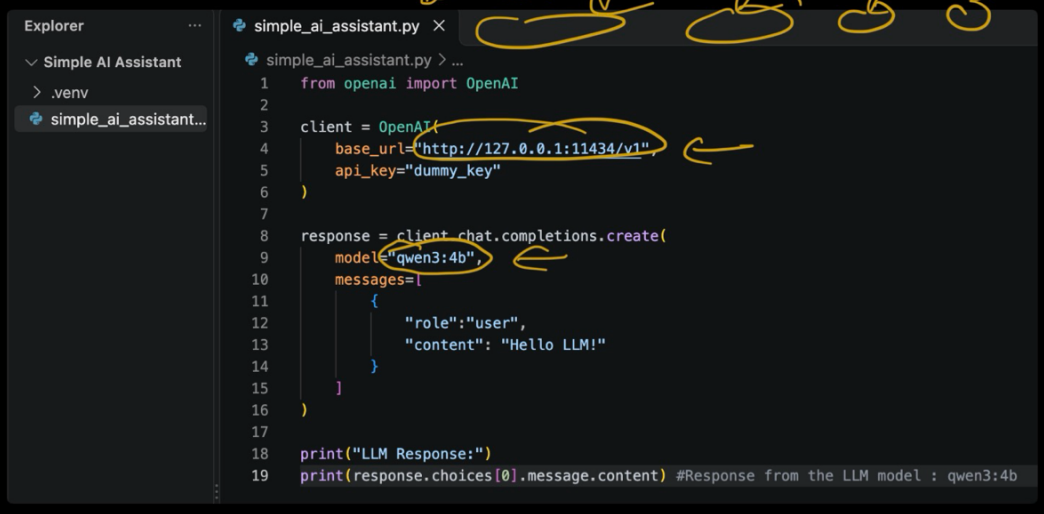
video-3

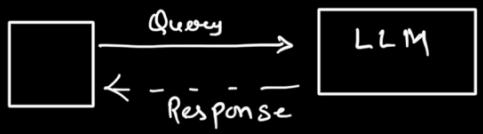


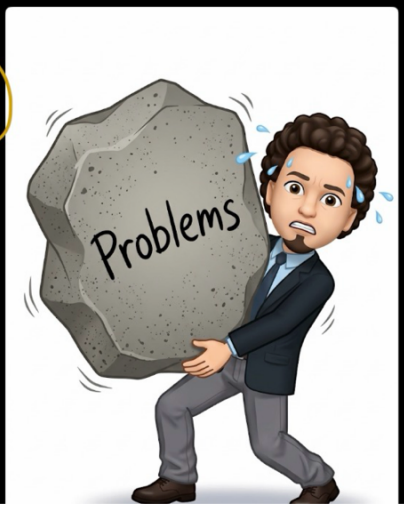


```
Explorer
Simple AI Assistant
.venv
simple_ai_assistant...
simple_ai_assistant.py
simple_ai_assistant.py > ...
1  from openai import OpenAI
2
3  client = OpenAI(
4      base_url="http://127.0.0.1:11434/v1",
5      api_key="dummy_key"
6  )
7
8  response = client.chat.completions.create(
9      model="qwen3:4b",
10     messages=[
11         {
12             "role":"user",
13             "content": "Hello LLM!"
14         }
15     ]
16 )
17
18 print("LLM Response:")
19 print(response.choices[0].message.content) #Response from the LLM model : qwen3:4b
```

① Hard coded — URL, Key, model name.

② Assistant talks only once







Done.

③ "role" → ???

Hard coding ✗

base-url
api-Key
model

→

(-) They can change in future.
(-) Avoid changing the source file.
(-) Change required in every file.

Keep them in one separate place

What is .env file in Python?

.env

```
DB_HOST=localhost
DB_USER=myuser
DB_PASSWORD=secret
API_KEY=abc123
```

Loaded in Python using python-dotenv

Your Python App

```
from dotenv import load_dotenv
import os

load_dotenv()

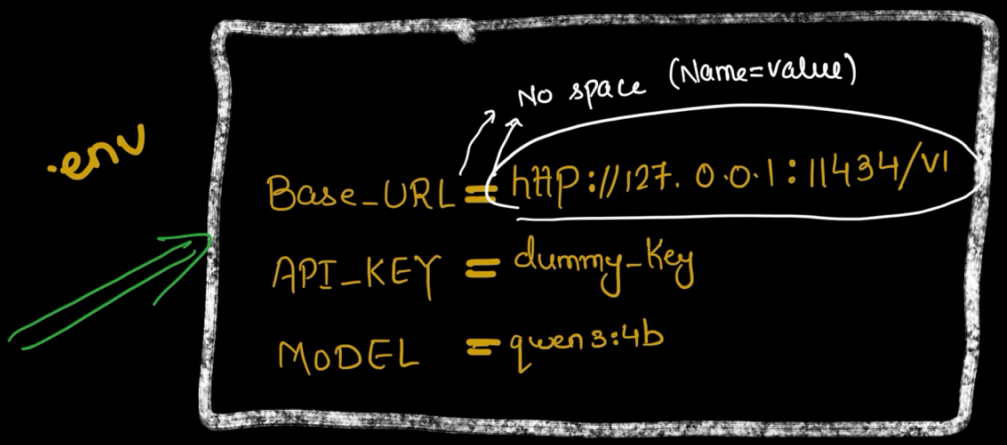
db_host = os.getenv("DB_HOST")
api_key = os.getenv("API_KEY")
```

A .env file is a simple text file that stores key-value pairs (environment variables).

It helps keep sensitive information like API keys, database credentials, etc. out of your code.

Why use it?

- Keeps secrets safe
- Cleaner code
- Easy to manage different environments (dev, prod, etc.)



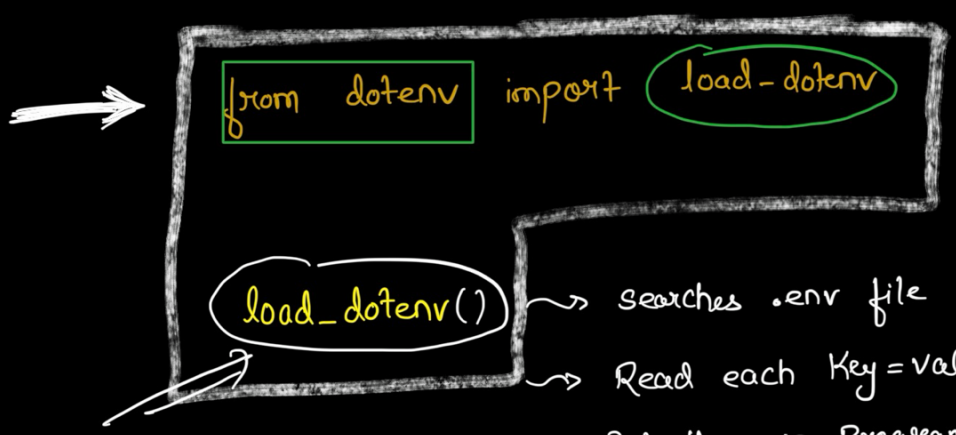
Industry standard →

.env

↓ environment → Because these become environment variables

hidden file

dotenv



It's loaded, How to read them in the code ???

```
import os
```
↳ Python's built-in module (os.py)

```
base_url = os.getenv("BASE_URL")
api_key = os.getenv("API_KEY")
```

model = os.getenv("MODEL")

Let's do Practical

What is "role" ??

```
response = client.chat.completions.create(
    model="qwen3:4b",
    messages=[
        {
            "role":"user",
            "content": "Hello LLM!"
        }
    ]
)
```

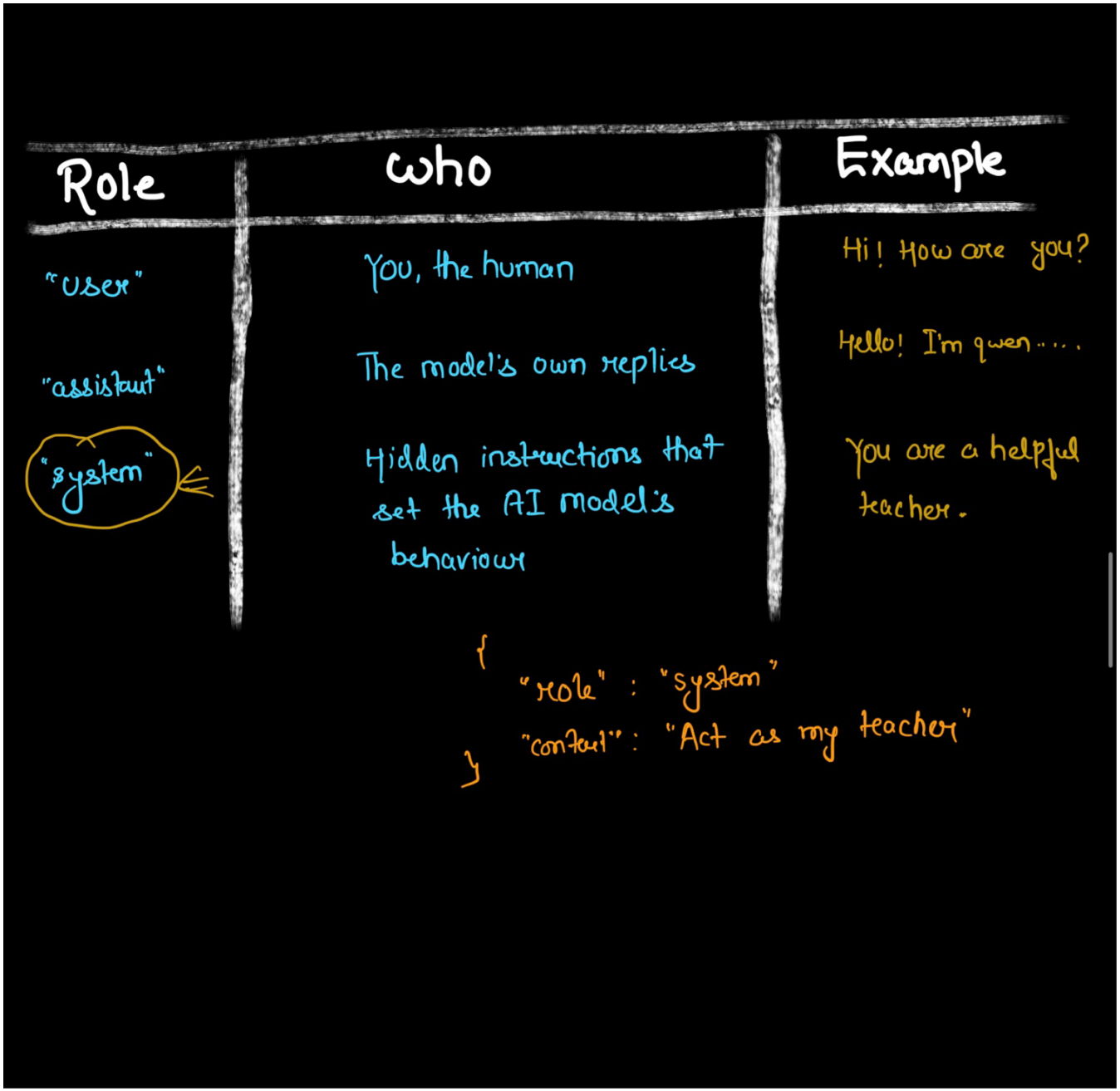




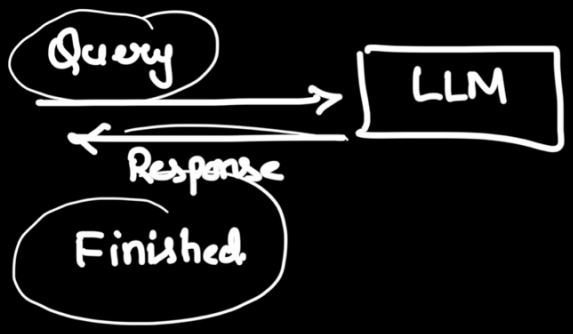
Chat

- Two or more speakers
- LLM needs to know who said what.
- "role" helps to identify the speaker.

```
{
  "role":    → who is speaking
  "content": → what they said
}
```


Role
who
Example
"user"
You, the human
Hi! How are you?
"assistant"
The model's own replies
Hello! I'm qwen....
"system"
Hidden instructions that set the AI model's behaviour
You are a helpful teacher.
{
"role" : "system"
"content" : "Act as my teacher"
}

Now, let's add a loop to make it a conversation.



we will add a loop so that it seems like a conversation (just like chatgpt etc.)

CONVERSATION

```
while True:
    user_input = input("\nYou:")

    response = client.chat.completions.create(
        model = os.getenv("MODEL")
        messages = [
            {
                "role" : "user",
                "content" : user_input
            }
        ]
    )

    print("Assistant: " + response.choices[0].message
                                .content)
```

"role" : assistant



Asst :